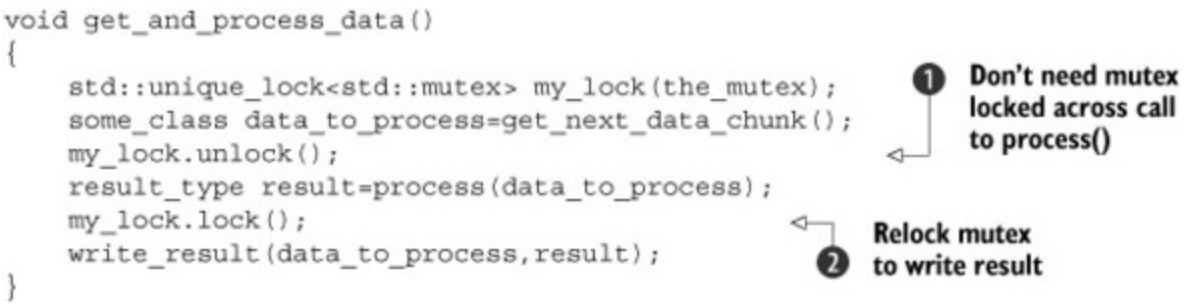


You don't need the mutex locked across the call to `process()`, so you manually unlock it before the call ❶ and then lock it again afterward ❷.

Hopefully it's obvious that if you have one mutex protecting an entire data structure, not only is there likely to be more contention for the lock, but also the potential for reducing the time that the lock is held is less. More of the operation steps will require a lock on the same mutex, so the lock must be held longer. This double whammy of a cost is thus also a double incentive to move toward finer-grained locking wherever possible.

As this example shows, locking at an appropriate granularity isn't only about the amount of data locked; it's also about how long the lock is held and what operations are performed while the lock is held. *In general, a lock should be held for only the minimum possible time needed to perform the required operations*. This also means that time-consuming operations such as acquiring another lock (even if you know it won't deadlock) or waiting for I/O to complete shouldn't be done while holding a lock unless absolutely necessary.

In listings 3.6 and 3.9, the operation that required locking the two mutexes was a swap operation, which obviously requires concurrent access to both objects. Suppose instead you were trying to compare a simple data member that was just a plain `int`. Would this make a difference? `int`s are cheap to copy, so you could easily copy the data for each object being compared while only holding the lock for that object and then compare the copied values. This would mean that you were holding the lock on each mutex for the minimum amount of time and also that you weren't holding one lock while locking another. The following listing shows a class `Y` for which this